Equality Saturation and E-Graphs: A Landscape of Modern Compiler Optimization

Compiler Theory & Practice (CPS2000) | Task 6 – Short paper on e-graphs

Kurt Abela (150106L) | kurt.abela.24@um.edu.mt

Introduction.....1
1. Understanding E-Graphs and Equality Saturation.....1
2. Historical Context and Evolution..... 2
3. E-Graphs in Real-World Compiling.....2
4. Future Trends and Directions Beyond 2026.....3
5. Reflection: E-Graphs vs. ResIR Task 5.....3
Bibliography.....4

Introduction

Traditional compiler optimization often suffers from the "phase ordering problem," where the sequence of applied transformations limits the potential for finding the most efficient code. This paper explores the emergence of e-graphs and equality saturation as a solution to this challenge. By representing an entire space of equivalent programs simultaneously, e-graphs allow compilers to bypass destructive rewrites in favor of a holistic, cost-based selection process. This piece examines the historical context, real-world applications, and the future trajectory of this technology.

1. Understanding E-Graphs and Equality Saturation

An **e-graph** (equivalence graph) is a specialized data structure designed to represent a large set of equivalent expressions compactly. Unlike a standard Abstract Syntax Tree (AST), which represents a single version of a program, an e-graph groups equivalent nodes into **e-classes**.

The Equality Saturation Workflow:

1. **Initial Representation:** The input program is loaded into the e-graph.
2. **Saturation:** The compiler applies rewrite rules (e.g., $a + 0 \rightarrow a \; or \; x \ll 1 \rightarrow x * 2$). Crucially, these rewrites are **non-destructive**; instead of replacing the old expression, the new version is added to the same e-class.

3. **Equality Saturation:** This process continues until no new equivalents can be added (the graph is "saturated") or a resource limit is reached.
4. **Extraction:** A cost function (e.g., minimizing instruction count or latency) is applied to the graph to select the single most efficient representation from the e-classes.

This workflow targets the **Phase Ordering Problem**, a classic compiler challenge where applying Optimization A before Optimization B might prevent B from ever triggering. Equality saturation effectively explores all paths simultaneously, ensuring no optimization opportunity is "hidden" by a previous choice.

2. Historical Context and Evolution

The theoretical foundation of equality saturation was pioneered by **Tate et al. (2009)**, who introduced the concept as a way to move away from the "search-based" or "greedy" heuristics of traditional optimizers.

For nearly a decade, the technique was considered too slow for production compilers due to the memory overhead of storing massive e-graphs. However, the landscape changed with the introduction of the **egg library (Willsey et al., 2021)**. By implementing a technique called *deferred rebuilding*, egg allowed for significantly faster e-graph maintenance, making the technology viable for modern toolchains.

Other representative systems include:

- **Herbie:** A tool designed to improve the accuracy of floating-point expressions by exploring equivalent mathematical forms to find the one with the least rounding error.
- **SPORES:** A system that uses e-graphs to optimize linear algebra and relational queries, bridging the gap between database optimization and traditional compiling.

3. E-Graphs in Real-World Compiling

E-graphs are no longer purely academic; they are active components in industrial-grade rewriting tools and compilers.

- **Cranelift (aegraphs):** Cranelift, the code generator used in Wasmtime (a WebAssembly runtime), recently transitioned its mid-end optimizer to use "acyclic e-graphs" (aegraphs). This allows Cranelift to perform sophisticated instruction selection and simplification without the risk of infinite loops in its rewrite rules.

- **Glenside:** A compiler for hardware accelerators (like FPGAs and ASICs) that uses e-graphs to map high-level tensor operations (like those in machine learning) onto specific hardware primitives. E-graphs allow Glenside to find "hidden" hardware-efficient layouts that a human programmer might miss.

4. Future Trends and Directions Beyond 2026

As we move toward the late 2020s, e-graph research is shifting toward **Machine Learning (ML) Integration**. Rather than relying on hand-written cost functions for the "Extraction" phase, future compilers will likely use Reinforcement Learning to predict which e-class path will lead to the best hardware utilization.

Additionally, we expect to see **Cross-Language E-Graphs**, where e-graphs act as a universal intermediate representation. This would allow a single optimizer to apply high-level mathematical identities to code regardless of whether the source was C++, Rust, or Python.

5. Reflection: E-Graphs vs. ResIR Task 5

In Task 5 of the CPS2000 project, we utilized **Soufflé Datalog** for Live Variable Analysis. While Datalog is exceptionally efficient at fixed-point dataflow analysis (determining which variables are alive across blocks), it is less suited for **term rewriting**.

If we had utilized e-graphs on top of Task 5, the impact would have been significant in the **Math Expressions** of ResIR. While Task 5 can identify that a variable is "dead" and can be removed, an e-graph could have looked at an expression like `%d = bin sub %a, %b` inside a loop and discovered an equivalent, hardware-specific instruction or an algebraic simplification that remains invisible to a Datalog-based liveness check. In short: Task 5 finds *what* is used, but an e-graph finds the *best way* to compute it.

Bibliography

1. **Tate, R., Stepp, M., Tatlock, Z., & Lerner, S.** (2009). *Equality Saturation: A New Approach to Optimization*. POPL.
2. **Willsey, M., Nandi, C., Wang, Y. R., Flatt, O., Tatlock, Z., & Panchekha, P.** (2021). *egg: Fast and Extensible Equality Saturation*. POPL.
3. **Panchekha, P., Sanchez-Stern, A., Wilcox, J. R., & Tatlock, Z.** (2015). *Automatically Improving Accuracy for Floating Point Expressions*. PLDI (Herbie).
4. **Cranelift Documentation.** (2025/2026). *Aegraph Mid-end Optimizer Design*.
5. **ResIR Project Specification.** (2026). *Internal Documentation for COMPILER-CPS2000*.